

Practical: 3

Implementation of max-heap sort algorithm

11/08/2026

Aim: To Implement of max-heap sort algorithm.

Algorithm:

* Algorithm :-

Input :- Unsorted Array
Output :- Sorted Array

Step: 1 :- Start

Step: 2 :- Build a max Heap from the given array.

Step: 3 :- for $i = n/2 - 1$ down to 0, set
 $largest = i$;
 $left = 2 * i + 1$
 $right = 2 * i + 2$

Step: 4 :- If $left < n$ and $A[left] > A[largest]$, set $largest = left$

Step: 5 :- If $right < n$ and $A[right] > A[largest]$
 set $largest = right$.
 if ($largest \neq i$):
 swap ($A[i]$ and $A[largest]$)
 Apply the same heapify process to the Sub tree.

Step: 6 :- After building the Max Heap, the largest element is at $A[0]$.

Step: 7 :- for $j = n - 1$ down to 1:
 swap ($A[0]$, $A[j]$):
 $j--$

Step: 8 :- Heapify the root $A[0]$, set $largest = 0$
 $left = 2 * 0 + 1$;
 $right = 2 * 0 + 2$;

Step: 9 :- Compare root with children
 set $largest$ to the index containing the largest value

Step: 10 :- repeat step 4 until $j = 1$

Step: 11 :- The array is now sorted in ascending order

Step: 12 :- Stop

Program:

Max-heap sort algorithm

```
#include <iostream>
using namespace std;
```

```
void max_heapify(int arr[], int n, int i)
{
    int largest = i;

    int l = 2 * i + 1;
```

```
int r = 2 * i + 2;

// Check left child
if (l < n && arr[l] > arr[largest])
{
    largest = l;
}

// Check right child
if (r < n && arr[r] > arr[largest])
{
    largest = r;
}

// If largest is not the root
if (largest != i)
{
    int t = arr[i];
    arr[i] = arr[largest];
    arr[largest] = t;

    // Heapify the affected subtree
    max_heapify(arr, n, largest);
}
}

void heapSort(int arr[], int n)
{
    // Build Max Heap
    for (int i = n / 2 - 1; i >= 0; i--)
    {
        max_heapify(arr, n, i);
    }

    // Heap Sort
    for (int j = n - 1; j > 0; j--)
    {
        // Move largest element to the end
        int t = arr[0];
        arr[0] = arr[j];
        arr[j] = t;

        // Heapify remaining heap
        max_heapify(arr, j, 0);
    }
}
```

```
void display(int arr[], int n)
{

    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }

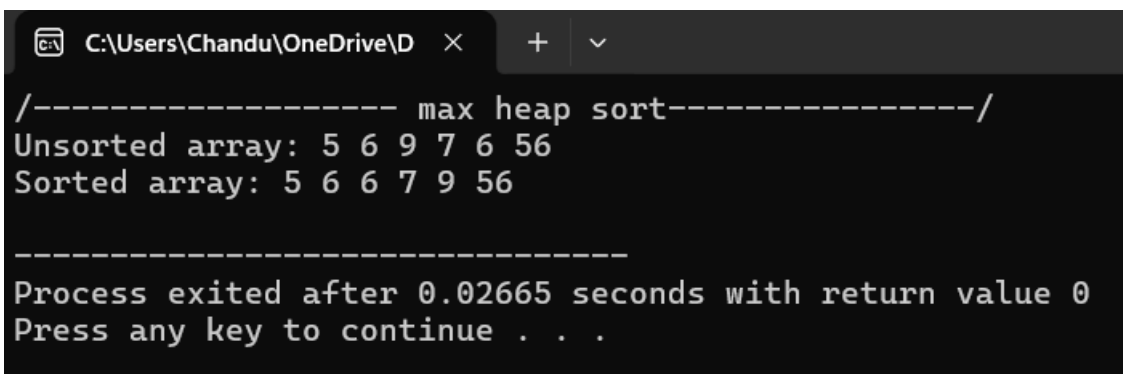
    cout << endl;
}

int main()
{
    int arr[] = {5, 6, 9, 7, 6, 56};
    cout<<"/----- max heap sort-----/"<<endl;
    int size = sizeof(arr) / sizeof(arr[0]);
    cout << "Unsorted array: ";
    display(arr, size);

    heapSort(arr, size);

    cout << "Sorted array: ";
    display(arr, size);

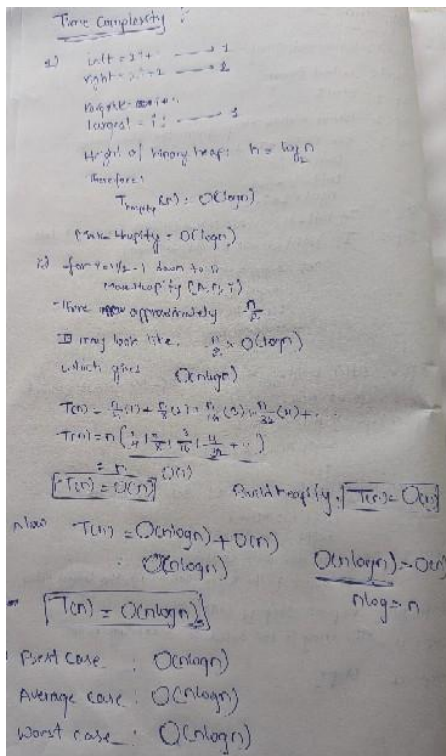
    return 0;
}
```

Output:

```
C:\Users\Chandu\OneDrive\D >
/----- max heap sort-----/
Unsorted array: 5 6 9 7 6 56
Sorted array: 5 6 6 7 9 56

-----
Process exited after 0.02665 seconds with return value 0
Press any key to continue . . .
```

Time Complexity:



Conclusion:

The Implementation of max-heap sort algorithm and the program of the max-heap sort Successfully got executed.

The time complexities of the max-heap sort algorithm in three cases best, average, worst is $O(n \log n)$.